

B.E in INFORMATION TECHNOLOGY
2ND YEAR, 1ST SEMESTER EXAMINATION, 2020
OBJECT ORIENTED PROGRAMMING

Time: 3 hours

Full Marks: 100

Question No. 1 and 2 are compulsory

CO1

1. a) Distinguish between normal function and inline function. State the cases when a function is not executed as inline even if it is declared as **inline**.
- b) Discuss two uses of **this** operator with valid code snippets. Distinguish between **this->** and **(*this)**, by their usage.
- c) Assume a function with the signature as **void sum(int a, int b, int c)**. Show three proper ways of supplying default parameter values in the given prototype and their corresponding function call statements.
- d) What do you mean by the reference of a variable? Show with a suitable example of code where a function must have to return the reference of a variable.

[(2+2)+(3+2)+(3+3)+(2+3)=20]

CO2

2. a) Consider the following code snippet having the definition of a class **Complex**. Define all the methods outside the class definition and show how they are invoked from the **main()** function.

```
class Complex
{ float real, imag;
public:
    void setData (float r, float i);
    friend Complex sum (Complex, Complex);
    void subtract (Complex);
    Complex multiply (Complex);
    void show();
};
```

- b) Assume two classes **ABC** and **XYZ** have a private data member each of their own. Initialize their values with suitable constructors. Show (with proper code snippet) how the values of those two member variables can be swapped by only one function.
- c) Consider the following class **Number**. Point out the illegal statements from the entire code with justification.

```
class Number
{int num;
public:
    Number();
    Number (int);
    Number (Number);
    static void show()
    { cout<<"Within show method";
    }
    friend int sum (Number n)
    { return (num+n.num);
    }
    int add (Number n)
    {return( num+n.num);
    }
};
```

```
int main()
{Number n1, n2(10);
n1.num=20;
Number n3(n2);
show();
cout<<n2.sum (n1);
cout<<n3.add(n2);
}
```

d) Assume a class **Sample** having a static member function **fun()** and a non-static member function **invoke()**. Show how the **fun()** function can be called from the body of **invoke()** function and from the **main()** function. Show with the same code, how a constant data member of the class can be initialized with some value.

[7+5+4+(3+1)=20]

C03

3. a) Consider a class **Sample** having a constructor that takes two parameters. Show 2 ways of supplying default parameter values in the constructor. Also show on creating objects, how they can be invoked.

b) Assume a class **Base** with a parameterized constructor and a destructor. Create a derived class **Child** inheriting the **Base** class having its own constructor and destructor. Show (with code snippet) how the constructor of the **Base** class is invoked through **Child** class. State in what order these constructors and destructors of the two classes are invoked. Discuss in this connection, the drawback of such destructor and how this drawback can be overcome.

c) Assume a class **Base** having a *protected* data member *x* and the pre-increment (++) operator overloaded function. From this class, a class **Child** has been inherited in public mode with the post-decrement (--) operator overloaded function. Now show how these operator overloaded functions are invoked through the **Child** class object.

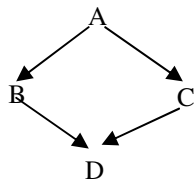
Assume both the pre-increment and post-decrement operator overloaded functions to be **constant** member functions. By setting the initial value of the data member *x* to 10 through **Child** class constructor, discuss the output of the program according to the sequence in which the function call statements appear in the **main()** method.

d) Distinguish between *early binding* and *late binding* with suitable example of code snippets.

[(2+1)+(2+1+4)+6+4=20]

Or,

4. a) What kind of inheritance does the following diagram indicate?



Suppose the class **A** has a parameterized and a default constructor. Each of the two derived classes **B** and **C** have their own parameterized constructors. Show (with code snippet and output) how the parameterized constructor of class **A** can be invoked through the object of class **D**. Hence in this connection, discuss the drawback of such type of inheritance. Explain (with code and output) how this drawback can be overcome.

b) Suppose a class **Base** has a member function **show()**. Two classes **Child1** and **Child2** have been derived from it and override the **show()** function. Show with a suitable code how the derived class versions of the **show()** function can be invoked without the direct involvement of the derived class objects. How is this phenomenon known as? Show how this code can be modified in the slightest possible way in order to invoke the **Base** class version of the **show()** function without direct involvement of the **Base** class object.

c) Suppose two classes **Base1** and **Base2** have a public member function **display()** each within them. A **Child** class has been inherited from these two classes. Show (with proper code) how the **Base2** class version of the **display()** function can be invoked through the **Child** class object. What type of inheritance does it refer?

[(1+4+2+3)+(3+1+2)+(3+1)=20]

C04:

5. a) Consider the following **Sample** class. Complete it with appropriate code to perform the operations as directed in the **main()** function. Discuss the output.

```

class Sample
{ int s;
//define suitable constructors
and functions
};
  
```

```

int main()
{Sample s1(10), s2(5),s3;
 s3=s2++;
 s1*=s2;
 int x=s1;
 int y=s2;
 Sample s4(x+y);
 cout<<s4;
}
  
```

b) Complete the following classes with appropriate code to perform the operations mentioned in the *main()* function. Discuss the output. The order of definition of the classes must be maintained as indicated below.

```

class Test
{ int t;
//define suitable constructors and member functions only
};

class Sample
{ int s;
//define suitable constructors and member functions only
};

int main()
{ Sample S1(5);
  Sample S2(S1);
  Test T1,T2;
  T2=T1(10);
  S1=T2;
  Sample S3(++S1)+S2;
  S3->display();
}

```

c) Consider a namespace NS having the declaration of a function **display()** and the definition of a class **Sample**. This **Sample** class is also having the declaration of a member function with the same signature as that of the **display()** method. Now show how these two versions of the **display()** method can be defined outside the namespace definition.

d) Discuss when **dynamic_cast** operator works correctly for inherited classes.

[7+8+3+2=20]

Or,

6. a) Consider the following **Sample** class. Complete it with appropriate code to perform the operations as directed in the *main()* function. Discuss the output.

```

class Sample
{ int s;
//define suitable constructors
and friend functions only
};

int main()
{Sample s1(5), s2;
 s2=s1++;
 cout<<s2;
 cout<<s1;
 s2=10+s1;
 Sample s3(--s1)*s2;
 cout<<s3;
}

```

b) Distinguish between **free()** function and **delete** operator.

Assume a class **Student** having private data members **name**, **roll** and **cgpa**. Now declare a pointer of this class type and create an array of objects of 50 students overloading the **new** operator. Call suitable constructors and member methods through that pointer for initializing the data members and displaying them. After performing all the operations, overload the **delete** operator to de-allocate the array.

c) Consider the following namespaces. Discuss the output with suitable explanation.

```

namespace ns1
{ int x=2, y=3;
void show(int a, int b)
{ x=a+b;
  cout<<"x+y="<<x+y;
}
namespace ns2
{ int x,y;
void show(int a, int b)
{ x=a; y=b;
cout<<x+y;
}
}
//end of ns2
//end of ns1

int main()
{
  ns1::show(10,20);
  ns1::x=5;
  ns1::y=6;
  ns1::ns2::show(ns1::x,ns1::y);
  using namespace ns1::ns2;
  show(1,2);
  show(ns1::x, y);
  show(x,ns1::y);
  ns1::show(x,y);
  return 0;
}

```

[8+(1+5)+6=20]

CO5:

7. a) Discuss the need of defining specialized function template.

Define a template function ***cube()*** that takes a parameter of type T and returns its cube. Now define a specialized version of this template function that takes a string type parameter and concatenates the input string three times and returns it. *Example: input string-> "JU", output string-> "JUJUJU".*

b) Discuss explicit and implicit call of template functions with suitable examples.

c) "While throwing an exception, the control is transferred to the catch block and then all objects created inside the closing try block are destructed"- justify the validity of the statement with proper code.

d) Assume a class ***Student*** having *roll number*, *name* and *cgpa* of each student as its data members. Define suitable member functions in the class to take the input of the data members from user. Create an array of 50 ***Student*** objects. Write a complete C++ program to open a file and write the records of these students in that file.

[(2+5)+3+3+7=20]

Or,

8. a) Consider a function template with specification as ***template <typename T, int count=3>***.

Now define a function ***void iterate (T x)*** that takes a parameter x of type T and displays it ***count*** number of times.

Pick up the correct function call statements (with outputs) from the following list with proper justification.

iterate <int, 4> (5);

iterate<float> (10.5);

iterate<> (10);

iterate (10);

iterate(10,4);

iterate<4> (10);

iterate<float,4>(x,n); //x is a float and n is an integer variable whose value have been taken

iterate<float, 4>(10, n); // n is an integer variable whose value has been taken

b) Assume a class template having the template specification as ***template<class X, class Y=int>***. Define suitable constructors to initialize the member variables of the class and also show how the corresponding objects are created.

c) Illustrate (with a suitable code) how exceptions of integer, float, character and string types are thrown from one try block and caught in different catch blocks. Show how to replace these multiple catch blocks by a single one.

d) Write a complete C++ program to open a file "a.txt" and copy all its contents into another file "b.txt".

[(2+4)+3+(3+2)+6=20]

Course Outcomes:

CO1: Recognise and **illustrate** the procedural enhancements of object-oriented programming languages over procedural languages.

CO2: Explain, illustrate and **recognise** the basic features of classes and objects.

CO3: Illustrate the extended features of OOP (Inheritance, Polymorphism) and **apply** them in practical problem solving.

CO4: Explain and **illustrate** RTTI, Namespace and Operator overloading.

CO5: Demonstrate I/O, exception handling and generic programming.